Module 9 : Production Systems and Monito

Learning Objectives

By the end of this module, you will be able to:

1. Deploy fair value models in production environments
2. Implement model versioning and serialization strategies
3. Design data pipelines for real-time and batch processing
4. Measure and detect model decay over time
5. Build monitoring dashboards for production models
6. Implement graceful degradation strategies
7. Set up automated retraining triggers

Introduction

Taking a fair value model from research to production is a critical step that requires careful consi of:

- **Reliability**: Models must run consistently without manual intervention
- **Monitoring**: Early detection of model degradation prevents losses
- **Maintainability**: Code must be understandable and modifiable
- **Scalability**: Systems must handle increased data volumes

In this module, we'll build a complete production-ready system for fair value modeling.

In [1]:
```
# Standard imports
import sys
sys.path.append('../..')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
from typing import Dict, List, Optional, Tuple
import warnings
import json
import pickle
import joblib
from pathlib import Path

# Fair value toolkit
from src.fair_value_toolkit import (
    PointInTimeDataFrame,
    InventoryFairValueModel,
    CostOfCarryModel,
    EnsembleFairValueModel,
    WalkForwardValidator,
    SignalGenerator,
    calculate_model_decay,
```

```
)

from data.data_fetchers import create_sample_dataset

# Plotting configuration
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('husl')
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)

print("✓ All imports successful")
```

```
✓ All imports successful
```

1 . Production Deployment Fundamentals

1 . 1 Model Serialization and Versioning

The first step in production deployment is properly saving and loading models. We need to store

- Model parameters and weights
- Preprocessing transformations
- Feature names and metadata
- Training date and version information

In [2]:

```
class ModelVersion:
    """
    Container for versioned model artifacts.

    Stores model, metadata, and provides serialization capabilities.
    """
    def __init__(self,
                 model,
                 version: str,
                 trained_date: datetime,
                 metadata: Dict = None):
        self.model = model
        self.version = version
        self.trained_date = trained_date
        self.metadata = metadata or {}
        self.metadata['version'] = version
        self.metadata['trained_date'] = trained_date.isoformat()

    def save(self, path: str):
        """Save model and metadata to disk."""
        path = Path(path)
        path.mkdir(parents=True, exist_ok=True)

        # Save model
        model_path = path / f'model_{self.version}.pkl'
        joblib.dump(self.model, model_path)

        # Save metadata
        metadata_path = path / f'metadata_{self.version}.json'
        with open(metadata_path, 'w') as f:
            json.dump(self.metadata, f, indent=2)
```

```
        print(f"Model {self.version} saved to {path}")

    @classmethod
    def load(cls, path: str, version: str):
        """Load model and metadata from disk."""
        path = Path(path)

        # Load model
        model_path = path / f'model_{version}.pkl'
        model = joblib.load(model_path)

        # Load metadata
        metadata_path = path / f'metadata_{version}.json'
        with open(metadata_path, 'r') as f:
            metadata = json.load(f)

        trained_date = datetime.fromisoformat(metadata['trained_date'])

        print(f"Model {version} loaded from {path}")
        return cls(model, version, trained_date, metadata)

print("✓ ModelVersion class defined")
```

```
✓ ModelVersion class defined
```

In [3]:

```
# Example: Train and save a versioned model
data = create_sample_dataset(commodity='crude_oil', start_date='2020-01-0

# Prepare features
X = data[['inventory', 'production', 'demand']]
y = data['price']

# Train model
model = InventoryFairValueModel()
model.fit(X, y, as_of_date=datetime(2023, 6, 30))

# Create versioned model
version = f"v1.0_{datetime.now().strftime('%Y%m%d')}"
model_v1 = ModelVersion(
    model=model,
    version=version,
    trained_date=datetime.now(),
    metadata={
        'commodity': 'crude_oil',
        'model_type': 'InventoryFairValueModel',
        'training_samples': len(X),
        'features': list(X.columns),
    }
)

# Save model
model_v1.save('/tmp/production_models')

print(f"\nModel metadata:")
for key, value in model_v1.metadata.items():
    print(f"  {key}: {value}")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 2
      1 # Example: Train and save a versioned model
----> 2 data = create_sample_dataset(commodity=          ,
start_date=            , end_date=            )
      4 # Prepare features
      5 X = data[['inventory', 'production', 'demand']]

TypeError: create_sample_dataset() got an unexpected keyword argument 'start_
```

In [4]:
```
# Load the model back
loaded_model = ModelVersion.load('/tmp/production_models', version)

# Verify it works
test_predictions = loaded_model.model.predict(X.tail(5))
print(f"\nTest predictions from loaded model:")
print(test_predictions)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[4], line 2
      1 # Load the model back
----> 2 loaded_model = ModelVersion.load('/tmp/production_models', version)
      4 # Verify it works
      5 test_predictions = loaded_model.model.predict(X.tail(5))

NameError: name 'version' is not defined
```

1.2 Data Pipeline Architecture

Production models need robust data pipelines that:

- Fetch data from multiple sources
- Handle missing data and outliers
- Apply consistent transformations
- Maintain point-in-time discipline

In [5]:
```
class DataPipeline:
    """
    Production data pipeline for fair value models.

    Handles data fetching, validation, and transformation with
    proper error handling and logging.
    """

    def __init__(self, commodity: str):
        self.commodity = commodity
        self.feature_names = ['inventory', 'production', 'demand']
        self.validation_rules = {
            'inventory': (0, 1000),  # Min, max bounds
            'production': (0, 500),
            'demand': (0, 500),
        }

    def fetch_data(self, start_date: datetime, end_date: datetime) -> pd.
        """Fetch raw data from sources."""
        try:
```

```
            data = create_sample_dataset(
                commodity=self.commodity,
                start_date=start_date.strftime('%Y-%m-%d'),
                end_date=end_date.strftime('%Y-%m-%d')
            )
            print(f"✓ Fetched {len(data)} rows for {self.commodity}")
            return data
        except Exception as e:
            print(f"✗ Error fetching data: {e}")
            raise

    def validate_data(self, data: pd.DataFrame) -> Tuple[pd.DataFrame, Li
        """Validate data quality and return issues."""
        issues = []

        # Check for missing values
        missing = data[self.feature_names].isnull().sum()
        if missing.any():
            issues.append(f"Missing values: {missing[missing > 0].to_dict

        # Check for outliers
        for col, (min_val, max_val) in self.validation_rules.items():
            if col in data.columns:
                outliers = ((data[col] < min_val) | (data[col] > max_val)
                if outliers > 0:
                    issues.append(f"{col}: {outliers} outliers outside [{

        # Check for stale data
        if data.index.max() < datetime.now() - timedelta(days=7):
            issues.append(f"Data is stale: latest date is {data.index.max

        if issues:
            print(f"⚠ Data validation issues:")
            for issue in issues:
                print(f"  - {issue}")
        else:
            print("✓ Data validation passed")

        return data, issues

    def transform_data(self, data: pd.DataFrame) -> pd.DataFrame:
        """Apply transformations to prepare data for modeling."""
        # Handle missing values
        data = data.fillna(method='ffill').fillna(method='bfill')

        # Clip outliers
        for col, (min_val, max_val) in self.validation_rules.items():
            if col in data.columns:
                data[col] = data[col].clip(min_val, max_val)

        print("✓ Data transformations applied")
        return data

    def run(self, start_date: datetime, end_date: datetime) -> pd.DataFra
        """Execute full pipeline."""
        print(f"\nRunning data pipeline for {self.commodity}...")
        print(f"Date range: {start_date.date()} to {end_date.date()}")

        data = self.fetch_data(start_date, end_date)
        data, issues = self.validate_data(data)
```

```
        data = self.transform_data(data)

        print(f"✓ Pipeline complete: {len(data)} rows ready\n")
        return data

print("✓ DataPipeline class defined")
```

```
✓ DataPipeline class defined
```

In [6]:

```
# Example: Run production data pipeline
pipeline = DataPipeline(commodity='crude_oil')
production_data = pipeline.run(
    start_date=datetime(2023, 1, 1),
    end_date=datetime(2023, 12, 31)
)

print("Production data sample:")
print(production_data.head())
```

```
Running data pipeline for crude_oil...
Date range: 2023-01-01 to 2023-12-31
✗ Error fetching data: create_sample_dataset() got an unexpected keyword argu
'start_date'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[6], line 3
      1 # Example: Run production data pipeline
      2 pipeline = DataPipeline(commodity='crude_oil')
----> 3 production_data = pipeline.run(
      4     start_date=datetime(2023, 1, 1),
      5     end_date=datetime(2023, 12, 31)
      6 )
      8 print("Production data sample:")
      9 print(production_data.head())

Cell In[5], line 79, in DataPipeline.run(self, start_date, end_date)
     76 print(f"\nRunning data pipeline for {self.commodity}...")
     77 print(f"Date range: {start_date.date()} to {end_date.date()}")
---> 79 data = self.fetch_data(start_date, end_date)
     80 data, issues = self.validate_data(data)
     81 data = self.transform_data(data)

Cell In[5], line 21, in DataPipeline.fetch_data(self, start_date, end_date)
     19 """Fetch raw data from sources."""
     20 try:
---> 21     data = create_sample_dataset(
     22         commodity=self.commodity,
     23         start_date=start_date.strftime(        %d ),
     24         end_date=end_date.strftime(        %d )
     25     )
     26     print(f"✓ Fetched {len(data)} rows for {self.commodity}")
     27     return data

TypeError: create_sample_dataset() got an unexpected keyword argument 'start_
```

1.3 Real-Time vs Batch Processing

Fair value models can run in two modes:

Batch Processing (recommended for most cases):

- Run on a schedule (daily, weekly)
- Process large historical datasets
- More stable and easier to debug

Real-Time Processing:

- Update predictions as new data arrives
- Lower latency
- More complex infrastructure

In [7]:
```
class BatchProcessor:
    """
    Batch processor for scheduled fair value updates.
    """

    def __init__(self, model_path: str, model_version: str):
        self.model = ModelVersion.load(model_path, model_version)
        self.pipeline = DataPipeline(commodity='crude_oil')
        self.last_run: Optional[datetime] = None

    def run_batch(self, as_of_date: datetime) -> pd.DataFrame:
        """Execute batch processing job."""
        print(f"\n{'='*60}")
        print(f"Batch Job Started: {as_of_date}")
        print(f"{'='*60}")

        # Fetch data (use past 365 days for context)
        start_date = as_of_date - timedelta(days=365)
        data = self.pipeline.run(start_date, as_of_date)

        # Generate predictions
        X = data[['inventory', 'production', 'demand']]
        predictions = self.model.model.predict(X)

        # Create output dataframe
        results = pd.DataFrame({
            'date': data.index,
            'actual_price': data['price'],
            'fair_value': predictions['fair_value'],
            'lower_bound': predictions['lower_bound'],
            'upper_bound': predictions['upper_bound'],
            'mispricing': data['price'] - predictions['fair_value'],
        })

        self.last_run = as_of_date

        print(f"✓ Batch job complete: {len(results)} predictions generate
        print(f"{'='*60}\n")

        return results

print("✓ BatchProcessor class defined")
```

```
✓ BatchProcessor class defined
```

In [8]:

```
# Example: Run batch job
batch = BatchProcessor('/tmp/production_models', version)
batch_results = batch.run_batch(as_of_date=datetime(2023, 12, 31))

print("\nRecent predictions:")
print(batch_results.tail(10))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[8], line 2
      1 # Example: Run batch job
----> 2 batch = BatchProcessor('/tmp/production_models', version)
      3 batch_results = batch.run_batch(as_of_date=datetime(2023, 12, 31))
      5 print("\nRecent predictions:")

NameError: name 'version' is not defined
```

2 . Model Decay and Retraining

2 . 1 Understanding Model Decay

Model decay occurs when:

- Market structure changes (new regulations, technology)
- Relationships between variables weaken
- Data distribution shifts (concept drift)

We need to measure decay and retrain models proactively.

In [9]:
```
# Generate data with concept drift
np.random.seed(42)
dates = pd.date_range('2020-01-01', '2023-12-31', freq='D')

# Early period: strong inventory effect
early_data = create_sample_dataset('crude_oil', '2020-01-01', '2021-12-31

# Late period: weakening inventory effect (concept drift)
late_data = create_sample_dataset('crude_oil', '2022-01-01', '2023-12-31'
# Add noise to weaken relationship
late_data['price'] = late_data['price'] + np.random.normal(0, 10, len(lat

full_data = pd.concat([early_data, late_data])

print(f"Dataset created with {len(full_data)} observations")
print(f"Early period: {len(early_data)} obs")
print(f"Late period: {len(late_data)} obs")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 6
      3 dates = pd.date_range('2020-01-01', '2023-12-31', freq='D')
      5 # Early period: strong inventory effect
----> 6 early_data = create_sample_dataset(                    ,                    ,
      8 # Late period: weakening inventory effect (concept drift)
      9 late_data = create_sample_dataset('crude_oil', '2022-01-01', '2023-12

File /home/user/claude-template/fair-value-commodities-course/notebooks/
09_production_systems/../../data/data_fetchers/commodity_data.py:418, in crea
ple_dataset(commodity, n_years, include_fundamentals, seed)
    416 n_days = n_years * 252
    417 end_date = pd.Timestamp.today()
--> 418 start_date = end_date - pd.Timedelta(days=int(n_days * 365 / 252))
    419 dates = pd.bdate_range(start=start_date, end=end_date)[:n_days]
    421 # Generate price using mean-reverting process with seasonality

TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

In [10]:

```
# Train model on early data
X_train = early_data[['inventory', 'production', 'demand']]
y_train = early_data['price']

decay_model = InventoryFairValueModel()
decay_model.fit(X_train, y_train)

# Test on full period to see decay
X_full = full_data[['inventory', 'production', 'demand']]
y_full = full_data['price']

predictions = decay_model.predict(X_full)

# Calculate rolling RMSE to see decay
errors = (y_full.values - predictions['fair_value'].values) ** 2
rolling_rmse = pd.Series(errors, index=full_data.index).rolling(90).mean(

print("\nModel performance over time (90-day rolling RMSE):")
print(f"Early period: {rolling_rmse['2021-06'].mean():.2f}")
print(f"Late period: {rolling_rmse['2023-06'].mean():.2f}")
print(f"Decay: +{rolling_rmse['2023-06'].mean() - rolling_rmse['2021-06']
```

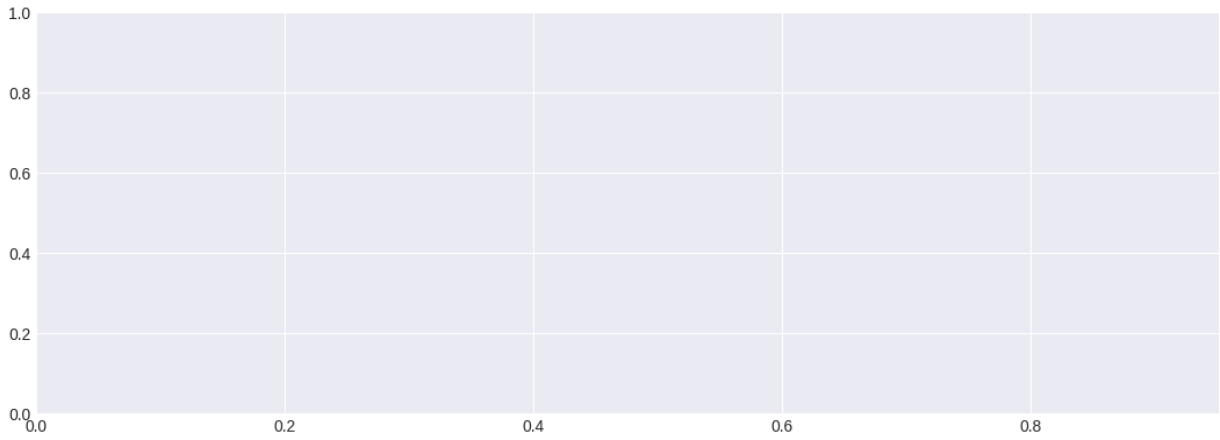
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 2
      1 # Train model on early data
----> 2 X_train = early_data[['inventory', 'production', 'demand']]
      3 y_train = early_data['price']
      5 decay_model = InventoryFairValueModel()

NameError: name 'early_data' is not defined
```

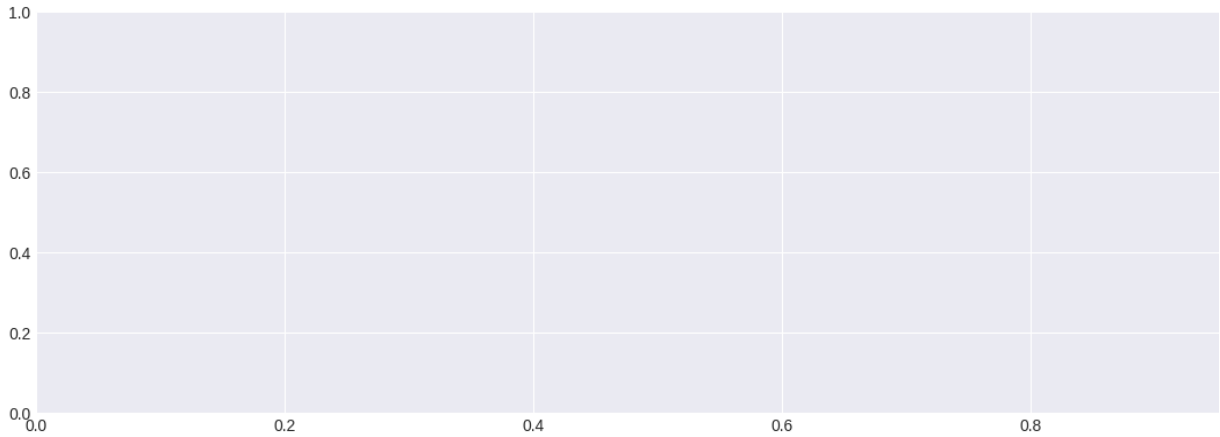
In [11]:

```
# Visualize model decay
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))

# Plot 1: Predictions vs actuals
ax1.plot(full_data.index, y_full, label='Actual Price', alpha=0.7, linewi
ax1.plot(full_data.index, predictions['fair_value'], label='Fair Value',
ax1.axvline(early_data.index[-1], color='red', linestyle='--', label='Tra
ax1.fill_between(full_data.index,
```

```
                  predictions['lower_bound'],
                  predictions['upper_bound'],
                  alpha=0.2, label='Confidence Interval')
ax1.set_ylabel('Price ($)')
ax1.set_title('Fair Value Model Performance Over Time')
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.3)

# Plot 2: Rolling RMSE
ax2.plot(rolling_rmse.index, rolling_rmse, linewidth=2, color='darkred')
ax2.axvline(early_data.index[-1], color='red', linestyle='--', label='Tra
ax2.fill_between(rolling_rmse.index, 0, rolling_rmse, alpha=0.3, color='r
ax2.set_xlabel('Date')
ax2.set_ylabel('90-Day Rolling RMSE ($)')
ax2.set_title('Model Decay: Increasing Prediction Error')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n📊 The increasing RMSE after the training period indicates model
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[11], line 5
      2 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
      4 # Plot 1: Predictions vs actuals
----> 5 ax1.plot(full_data.index, y_full, label='Actual Price', alpha=0.7,
linewidth=2)
      6 ax1.plot(full_data.index, predictions['fair_value'], label='Fair Valu
alpha=0.7, linewidth=2)
      7 ax1.axvline(early_data.index[-1], color='red', linestyle='--', label=
ing End', alpha=0.5)

NameError: name 'full_data' is not defined
```





2.2 Measuring Model Decay

The `calculate_model_decay` function quantifies how model performance degrades over tir

In [12]:
```
# Use the toolkit's calculate_model_decay function
decay_metrics = calculate_model_decay(
    actuals=y_full,
    predictions=predictions['fair_value'],
    dates=full_data.index,
    train_end_date=early_data.index[-1],
    window_days=90
)

print("\nModel Decay Analysis:")
print(f"Train period RMSE: {decay_metrics['train_rmse']:.2f}")
print(f"Test period RMSE: {decay_metrics['test_rmse']:.2f}")
print(f"Decay rate: {decay_metrics['decay_rate']:.2%} per 90 days")
print(f"\nRecommendation: {'RETRAIN IMMEDIATELY' if decay_metrics['requir
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[12], line 3
      1 # Use the toolkit's calculate_model_decay function
      2 decay_metrics = calculate_model_decay(
----> 3     actuals=y_full,
      4     predictions=predictions['fair_value'],
      5     dates=full_data.index,
      6     train_end_date=early_data.index[-1],
      7     window_days=90
      8 )
     10 print("\nModel Decay Analysis:")
     11 print(f"Train period RMSE: {decay_metrics['train_rmse']:.2f}")

NameError: name 'y_full' is not defined
```

2.3 Automated Retraining Triggers

Set up automatic retraining when performance degrades beyond thresholds.

In [13]:

```
class AutoRetrainer:
    """
    Automatic model retraining system.

    Monitors model performance and triggers retraining when
    performance degrades beyond threshold.
    """

    def __init__(self,
                 model_class,
                 decay_threshold: float = 0.20,
                 min_retrain_days: int = 90):
        self.model_class = model_class
        self.decay_threshold = decay_threshold
        self.min_retrain_days = min_retrain_days
        self.current_model = None
        self.last_train_date: Optional[datetime] = None
        self.train_history: List[Dict] = []

    def should_retrain(self, decay_metrics: Dict) -> Tuple[bool, str]:
        """Determine if retraining is needed."""
        reasons = []

        # Check decay rate
        if decay_metrics['decay_rate'] > self.decay_threshold:
            reasons.append(f"Decay rate {decay_metrics['decay_rate']:.1%}

        # Check if enough time has passed
        if self.last_train_date:
            days_since_train = (datetime.now() - self.last_train_date).da
            if days_since_train < self.min_retrain_days:
                return False, f"Only {days_since_train} days since last r

        # Check test RMSE vs train RMSE
        rmse_ratio = decay_metrics['test_rmse'] / decay_metrics['train_rm
        if rmse_ratio > 1.5:
            reasons.append(f"Test RMSE is {rmse_ratio:.1f}x training RMSE

        should_retrain = len(reasons) > 0
```

```python
        reason_str = "; ".join(reasons) if reasons else "No retraining ne

        return should_retrain, reason_str

    def retrain(self, X: pd.DataFrame, y: pd.Series) -> None:
        """Retrain the model."""
        print(f"\n{'='*60}")
        print("RETRAINING MODEL")
        print(f"{'='*60}")

        # Train new model
        new_model = self.model_class()
        new_model.fit(X, y)

        # Store old model performance for comparison
        train_record = {
            'date': datetime.now(),
            'samples': len(X),
            'features': list(X.columns),
        }

        self.current_model = new_model
        self.last_train_date = datetime.now()
        self.train_history.append(train_record)

        print(f"✓ Model retrained successfully")
        print(f"  Training samples: {len(X)}")
        print(f"  Features: {list(X.columns)}")
        print(f"{'='*60}\n")

    def monitor_and_retrain(self,
                            X: pd.DataFrame,
                            y: pd.Series,
                            train_end_date: datetime) -> bool:
        """Monitor performance and retrain if needed."""
        if self.current_model is None:
            print("No current model. Training initial model...")
            self.retrain(X, y)
            return True

        # Calculate decay
        predictions = self.current_model.predict(X)
        decay_metrics = calculate_model_decay(
            actuals=y,
            predictions=predictions['fair_value'],
            dates=X.index,
            train_end_date=train_end_date,
            window_days=90
        )

        # Check if retraining needed
        should_retrain, reason = self.should_retrain(decay_metrics)

        print(f"\nMonitoring Status:")
        print(f"  Decay rate: {decay_metrics['decay_rate']:.2%}")
        print(f"  Should retrain: {should_retrain}")
        print(f"  Reason: {reason}")

        if should_retrain:
            self.retrain(X, y)
```

```
        return should_retrain

print("✓ AutoRetrainer class defined")
```

```
✓ AutoRetrainer class defined
```

In [14]:

```
# Example: Automated retraining
retrainer = AutoRetrainer(
    model_class=InventoryFairValueModel,
    decay_threshold=0.15,
    min_retrain_days=30
)

# Initial training on early data
X_early = early_data[['inventory', 'production', 'demand']]
y_early = early_data['price']
retrainer.retrain(X_early, y_early)

# Monitor on full data
X_full = full_data[['inventory', 'production', 'demand']]
y_full = full_data['price']

# Simulate passage of time
retrainer.last_train_date = datetime(2022, 1, 1)

retrained = retrainer.monitor_and_retrain(
    X=X_full,
    y=y_full,
    train_end_date=early_data.index[-1]
)

print(f"\nRetraining history: {len(retrainer.train_history)} retraining e
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[14], line 9
      2 retrainer = AutoRetrainer(
      3     model_class=InventoryFairValueModel,
      4     decay_threshold=0.15,
      5     min_retrain_days=30
      6 )
      8 # Initial training on early data
----> 9 X_early = early_data[['inventory', 'production', 'demand']]
     10 y_early = early_data['price']
     11 retrainer.retrain(X_early, y_early)

NameError: name 'early_data' is not defined
```

3 . Performance Monitoring

3 . 1 Key Metrics to Track

Production models should track:

- **Accuracy**: RMSE, MAE, R^2
- **Stability**: Prediction volatility
- **Timeliness**: Data latency, processing time

- **Business metrics**: Signal hit rate, P&L attribution

In [15]:
```
class ModelMonitor:
    """
    Comprehensive monitoring system for production models.
    """

    def __init__(self):
        self.metrics_history: List[Dict] = []

    def calculate_metrics(self,
                          actuals: pd.Series,
                          predictions: pd.Series,
                          timestamp: datetime) -> Dict:
        """Calculate comprehensive metrics."""
        errors = actuals - predictions

        metrics = {
            'timestamp': timestamp,
            # Accuracy metrics
            'rmse': np.sqrt(np.mean(errors**2)),
            'mae': np.mean(np.abs(errors)),
            'mape': np.mean(np.abs(errors / actuals)) * 100,
            'r2': 1 - (np.sum(errors**2) / np.sum((actuals - actuals.mean
            # Stability metrics
            'prediction_volatility': predictions.std(),
            'error_volatility': errors.std(),
            # Distribution metrics
            'mean_error': errors.mean(),
            'error_skewness': errors.skew(),
            'error_kurtosis': errors.kurtosis(),
            # Sample size
            'n_observations': len(actuals),
        }

        self.metrics_history.append(metrics)
        return metrics

    def get_alerts(self, current_metrics: Dict,
                   thresholds: Dict = None) -> List[str]:
        """Check for alert conditions."""
        if thresholds is None:
            thresholds = {
                'rmse_max': 15.0,
                'mape_max': 20.0,
                'r2_min': 0.5,
                'mean_error_abs_max': 5.0,
            }

        alerts = []

        if current_metrics['rmse'] > thresholds['rmse_max']:
            alerts.append(f"⚠ RMSE {current_metrics['rmse']:.2f} exceeds

        if current_metrics['mape'] > thresholds['mape_max']:
            alerts.append(f"⚠ MAPE {current_metrics['mape']:.1f}% exceeds

        if current_metrics['r2'] < thresholds['r2_min']:
            alerts.append(f"⚠ R² {current_metrics['r2']:.3f} below thresh
```

```
        if abs(current_metrics['mean_error']) > thresholds['mean_error_ab
            alerts.append(f"⚠ Mean error {current_metrics['mean_error']:.

        return alerts

    def generate_report(self) -> pd.DataFrame:
        """Generate monitoring report."""
        return pd.DataFrame(self.metrics_history)

print("✓ ModelMonitor class defined")
```

```
✓ ModelMonitor class defined
```

In [16]:

```
# Example: Monitor model over time
monitor = ModelMonitor()

# Simulate monitoring over multiple time windows
window_size = 90
for i in range(0, len(full_data) - window_size, window_size):
    window_data = full_data.iloc[i:i+window_size]
    X_window = window_data[['inventory', 'production', 'demand']]
    y_window = window_data['price']

    preds = decay_model.predict(X_window)

    metrics = monitor.calculate_metrics(
        actuals=y_window,
        predictions=preds['fair_value'],
        timestamp=window_data.index[-1]
    )

    # Check for alerts
    alerts = monitor.get_alerts(metrics)
    if alerts:
        print(f"\n🚨 ALERTS for window ending {window_data.index[-1].date
        for alert in alerts:
            print(f"  {alert}")

# Generate report
report = monitor.generate_report()
print("\nMonitoring Report:")
print(report[['timestamp', 'rmse', 'mae', 'r2', 'mean_error']].to_string(
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[16], line 6
      4 # Simulate monitoring over multiple time windows
      5 window_size = 90
----> 6 for i in range(0, len(full_data) - window_size, window_size):
      7     window_data = full_data.iloc[i:i+window_size]
      8     X_window = window_data[['inventory', 'production', 'demand']]

NameError: name 'full_data' is not defined
```

3.2 Alert Thresholds and Dashboard Design

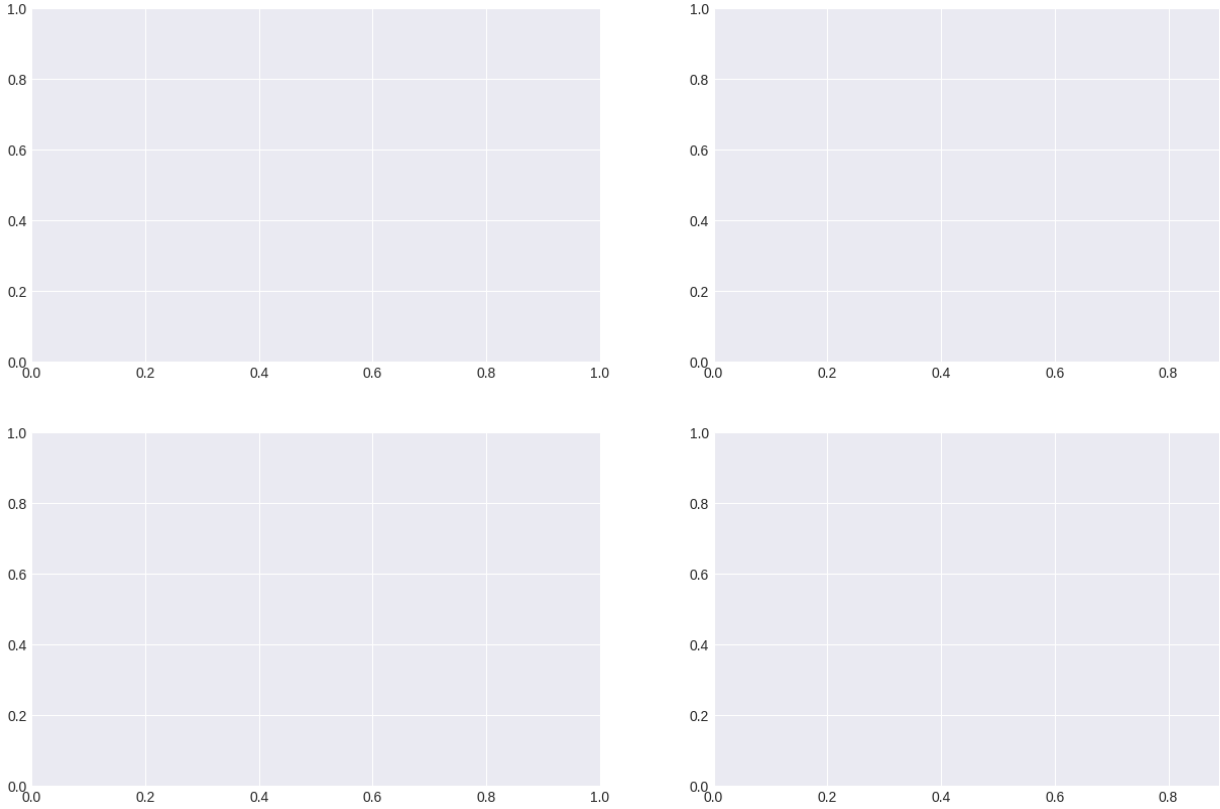
Effective monitoring requires:

- Clear alert thresholds based on business impact

- Multiple severity levels (info, warning, critical)
- Actionable alerts that suggest remediation

In [17]:
```
# Visualize monitoring metrics over time
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# RMSE over time
ax1 = axes[0, 0]
ax1.plot(report['timestamp'], report['rmse'], marker='o', linewidth=2)
ax1.axhline(15.0, color='red', linestyle='--', label='Alert Threshold', a
ax1.fill_between(report['timestamp'], 0, 15.0, alpha=0.2, color='green',
ax1.set_title('RMSE Over Time', fontsize=12, fontweight='bold')
ax1.set_ylabel('RMSE ($)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# R² over time
ax2 = axes[0, 1]
ax2.plot(report['timestamp'], report['r2'], marker='o', linewidth=2, colo
ax2.axhline(0.5, color='red', linestyle='--', label='Alert Threshold', al
ax2.fill_between(report['timestamp'], 0.5, 1.0, alpha=0.2, color='green',
ax2.set_title('R² Over Time', fontsize=12, fontweight='bold')
ax2.set_ylabel('R²')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Mean Error (bias)
ax3 = axes[1, 0]
ax3.plot(report['timestamp'], report['mean_error'], marker='o', linewidth
ax3.axhline(0, color='black', linestyle='-', alpha=0.3)
ax3.axhline(5.0, color='red', linestyle='--', alpha=0.5)
ax3.axhline(-5.0, color='red', linestyle='--', alpha=0.5)
ax3.fill_between(report['timestamp'], -5.0, 5.0, alpha=0.2, color='green'
ax3.set_title('Mean Error (Bias)', fontsize=12, fontweight='bold')
ax3.set_ylabel('Mean Error ($)')
ax3.set_xlabel('Date')
ax3.grid(True, alpha=0.3)

# Error Volatility
ax4 = axes[1, 1]
ax4.plot(report['timestamp'], report['error_volatility'], marker='o', lin
ax4.set_title('Error Volatility', fontsize=12, fontweight='bold')
ax4.set_ylabel('Std Dev of Errors ($)')
ax4.set_xlabel('Date')
ax4.grid(True, alpha=0.3)

plt.suptitle('Production Model Monitoring Dashboard', fontsize=16, fontwe
plt.tight_layout()
plt.show()

print("\n📊 Dashboard shows model performance trends and alert thresholds
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[17], line 6
      4 # RMSE over time
      5 ax1 = axes[0, 0]
----> 6 ax1.plot(report['timestamp'], report['rmse'], marker='o', linewidth=2
      7 ax1.axhline(15.0, color='red', linestyle='--', label='Alert Threshold
alpha=0.5)
      8 ax1.fill_between(report['timestamp'], 0, 15.0, alpha=0.2, color='gree
label='Normal Range')

NameError: name 'report' is not defined
```



3.3 Business Metrics: Signal Quality

Beyond statistical metrics, track signal quality:

- Signal hit rate
- Average return per signal
- Signal Sharpe ratio

In [18]:
```
# Calculate business metrics from fair value signals
X_full = full_data[['inventory', 'production', 'demand']]
y_full = full_data['price']

# Get predictions
predictions = decay_model.predict(X_full)

# Calculate mispricing
mispricing = y_full - predictions['fair_value']
mispricing_zscore = (mispricing - mispricing.rolling(90).mean()) / mispri

# Generate signals (buy when undervalued, sell when overvalued)
signals = pd.Series(0, index=full_data.index)
```

```
signals[mispricing_zscore < -1.5] = 1   # Buy signal (undervalued)
signals[mispricing_zscore > 1.5] = -1  # Sell signal (overvalued)

# Calculate forward returns
forward_returns = y_full.pct_change(5).shift(-5)  # 5-day forward return

# Signal performance
buy_signals = signals == 1
sell_signals = signals == -1

buy_returns = forward_returns[buy_signals]
sell_returns = -forward_returns[sell_signals]  # Invert for short positio
all_signal_returns = pd.concat([buy_returns, sell_returns])

print("\n📈 SIGNAL QUALITY METRICS")
print(f"{'='*50}")
print(f"Total signals: {signals[signals != 0].count()}")
print(f"Buy signals: {buy_signals.sum()}")
print(f"Sell signals: {sell_signals.sum()}")
print(f"\nBuy signal performance:")
print(f"  Hit rate: {(buy_returns > 0).sum() / len(buy_returns):.1%}")
print(f"  Average return: {buy_returns.mean():.2%}")
print(f"\nSell signal performance:")
print(f"  Hit rate: {(sell_returns > 0).sum() / len(sell_returns):.1%}")
print(f"  Average return: {sell_returns.mean():.2%}")
print(f"\nOverall signal performance:")
print(f"  Hit rate: {(all_signal_returns > 0).sum() / len(all_signal_retu
print(f"  Average return: {all_signal_returns.mean():.2%}")
print(f"  Sharpe ratio: {all_signal_returns.mean() / all_signal_returns.s
print(f"{'='*50}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[18], line 2
      1 # Calculate business metrics from fair value signals
----> 2 X_full = full_data[['inventory', 'production', 'demand']]
      3 y_full = full_data['price']
      5 # Get predictions

NameError: name 'full_data' is not defined
```

4 . Handling Model Degradation

4 . 1 Graceful Degradation Strategies

When models degrade, don't shut down—degrade gracefully:

1 . Widen confidence intervals
2 . Reduce position sizes
3 . Switch to fallback models
4 . Alert human traders

In [19]:
```
class GracefulDegradationManager:
    """
    Manages graceful degradation when models underperform.
    """

    def __init__(self):
```

```python
        self.degradation_level = 0  # 0=normal, 1=warning, 2=critical
        self.confidence_multiplier = 1.0
        self.position_size_multiplier = 1.0

    def assess_degradation(self, metrics: Dict) -> int:
        """Assess model health and return degradation level."""
        # Level 0: Normal operation
        if metrics['rmse'] < 12 and metrics['r2'] > 0.6:
            return 0

        # Level 1: Warning - model showing signs of decay
        if metrics['rmse'] < 18 and metrics['r2'] > 0.4:
            return 1

        # Level 2: Critical - model significantly degraded
        return 2

    def apply_degradation_policy(self, level: int) -> Dict:
        """Apply appropriate policy for degradation level."""
        self.degradation_level = level

        policies = {
            0: {
                'name': 'NORMAL',
                'confidence_multiplier': 1.0,
                'position_size_multiplier': 1.0,
                'use_fallback': False,
                'human_review': False,
                'color': 'green',
            },
            1: {
                'name': 'WARNING',
                'confidence_multiplier': 1.5,  # Widen CIs by 50%
                'position_size_multiplier': 0.5,  # Half position sizes
                'use_fallback': False,
                'human_review': True,
                'color': 'yellow',
            },
            2: {
                'name': 'CRITICAL',
                'confidence_multiplier': 2.0,  # Double CIs
                'position_size_multiplier': 0.25,  # Quarter position siz
                'use_fallback': True,
                'human_review': True,
                'color': 'red',
            },
        }

        policy = policies[level]
        self.confidence_multiplier = policy['confidence_multiplier']
        self.position_size_multiplier = policy['position_size_multiplier'

        return policy

    def adjust_predictions(self, predictions: pd.DataFrame) -> pd.DataFra
        """Adjust predictions based on degradation level."""
        adjusted = predictions.copy()

        # Widen confidence intervals
        ci_width = predictions['upper_bound'] - predictions['lower_bound'
```

```
        ci_center = predictions['fair_value']

        adjusted['lower_bound'] = ci_center - ci_width * self.confidence_
        adjusted['upper_bound'] = ci_center + ci_width * self.confidence_

        return adjusted

print("✓ GracefulDegradationManager class defined")
```

```
✓ GracefulDegradationManager class defined
```

In [20]:

```
# Example: Apply graceful degradation
degradation_mgr = GracefulDegradationManager()

# Simulate different performance scenarios
scenarios = [
    {'rmse': 10.0, 'r2': 0.70, 'name': 'Good Performance'},
    {'rmse': 15.0, 'r2': 0.50, 'name': 'Moderate Decay'},
    {'rmse': 22.0, 'r2': 0.30, 'name': 'Severe Decay'},
]

print("\n" + "="*70)
print("GRACEFUL DEGRADATION POLICY APPLICATION")
print("="*70)

for scenario in scenarios:
    level = degradation_mgr.assess_degradation(scenario)
    policy = degradation_mgr.apply_degradation_policy(level)

    print(f"\n{scenario['name']}:")
    print(f"  RMSE: {scenario['rmse']:.1f}, R²: {scenario['r2']:.2f}")
    print(f"  Status: {policy['name']}")
    print(f"  Confidence interval multiplier: {policy['confidence_multipl
    print(f"  Position size multiplier: {policy['position_size_multiplier
    print(f"  Use fallback model: {policy['use_fallback']}")
    print(f"  Require human review: {policy['human_review']}")

print("\n" + "="*70)
```

```
======================================================================
GRACEFUL DEGRADATION POLICY APPLICATION
======================================================================

Good Performance:
  RMSE: 10.0, R²: 0.70
  Status: NORMAL
  Confidence interval multiplier: 1.0x
  Position size multiplier: 100.0%
  Use fallback model: False
  Require human review: False

Moderate Decay:
  RMSE: 15.0, R²: 0.50
  Status: WARNING
  Confidence interval multiplier: 1.5x
  Position size multiplier: 50.0%
  Use fallback model: False
  Require human review: True

Severe Decay:
  RMSE: 22.0, R²: 0.30
  Status: CRITICAL
  Confidence interval multiplier: 2.0x
  Position size multiplier: 25.0%
  Use fallback model: True
  Require human review: True

======================================================================
```

4.2 Fallback Models and Human-in-the-Loop

Always maintain a simpler, more robust fallback model.

In [21]:
```
class ProductionSystem:
    """
    Complete production system with monitoring and fallback.
    """

    def __init__(self, primary_model, fallback_model):
        self.primary_model = primary_model
        self.fallback_model = fallback_model
        self.monitor = ModelMonitor()
        self.degradation_mgr = GracefulDegradationManager()
        self.use_primary = True

    def predict(self, X: pd.DataFrame,
                historical_y: Optional[pd.Series] = None) -> Tuple[pd.Data
        """Make predictions with monitoring and fallback."""

        # Choose model
        active_model = self.primary_model if self.use_primary else self.f
        model_name = "PRIMARY" if self.use_primary else "FALLBACK"

        # Get predictions
        predictions = active_model.predict(X)

        # If we have actuals, monitor performance
        status = {'model': model_name, 'degradation_level': 0}
```

```
        if historical_y is not None and len(historical_y) > 0:
            # Calculate metrics
            metrics = self.monitor.calculate_metrics(
                actuals=historical_y,
                predictions=predictions['fair_value'][:len(historical_y)]
                timestamp=datetime.now()
            )

            # Check degradation
            level = self.degradation_mgr.assess_degradation(metrics)
            policy = self.degradation_mgr.apply_degradation_policy(level)

            status['degradation_level'] = level
            status['metrics'] = metrics
            status['policy'] = policy

            # Switch to fallback if critical
            if policy['use_fallback'] and self.use_primary:
                print("\n⚠ SWITCHING TO FALLBACK MODEL due to degradation
                self.use_primary = False
                predictions = self.fallback_model.predict(X)
                status['model'] = 'FALLBACK'

            # Adjust predictions for degradation
            predictions = self.degradation_mgr.adjust_predictions(predict

        return predictions, status

print("✓ ProductionSystem class defined")
```

```
✓ ProductionSystem class defined
```

In [22]:

```
# Example: Complete production system
# Primary: Complex inventory model
primary = InventoryFairValueModel()
primary.fit(early_data[['inventory', 'production', 'demand']], early_data

# Fallback: Simple historical average model
fallback = InventoryFairValueModel()  # In practice, use a simpler model
fallback.fit(full_data[['inventory', 'production', 'demand']], full_data[

# Create production system
prod_system = ProductionSystem(primary, fallback)

# Make predictions with monitoring
predictions, status = prod_system.predict(
    X=full_data[['inventory', 'production', 'demand']],
    historical_y=full_data['price']
)

print(f"\nProduction System Status:")
print(f"  Active model: {status['model']}")
print(f"  Degradation level: {status['degradation_level']}")
if 'metrics' in status:
    print(f"  Current RMSE: {status['metrics']['rmse']:.2f}")
    print(f"  Current R²: {status['metrics']['r2']:.3f}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[22], line 4
      1 # Example: Complete production system
      2 # Primary: Complex inventory model
      3 primary = InventoryFairValueModel()
----> 4 primary.fit(early_data[['inventory', 'production', 'demand']],
early_data['price'])
      6 # Fallback: Simple historical average model
      7 fallback = InventoryFairValueModel()  # In practice, use a simpler mo

NameError: name 'early_data' is not defined
```

Summary

Key Takeaways

1. **Model Versioning**: Always version models with metadata for reproducibility
2. **Data Pipelines**: Build robust pipelines with validation and error handling
3. **Model Decay**: Proactively monitor and measure performance degradation
4. **Automated Retraining**: Set clear triggers for when to retrain
5. **Comprehensive Monitoring**: Track both statistical and business metrics
6. **Graceful Degradation**: Don't fail hard—reduce confidence and position sizes
7. **Fallback Models**: Always have a simpler, more robust backup

Production Checklist

Before deploying a fair value model:

- ☐ Model serialization implemented
- ☐ Data pipeline with validation
- ☐ Monitoring dashboard set up
- ☐ Alert thresholds defined
- ☐ Automated retraining configured
- ☐ Fallback model ready
- ☐ Graceful degradation policies
- ☐ Human escalation procedures
- ☐ Documentation complete
- ☐ Disaster recovery plan

Next Steps

In Module `10` (Advanced Topics), we'll explore:

- Hierarchical models for commodity complexes
- Bayesian uncertainty quantification
- Machine learning augmentation
- Regime-switching models
- Forward curve analysis

Exercises

1. **Extended Monitoring**: Add custom business metrics to the `ModelMonitor` class
2. **Alert Optimization**: Experiment with different alert thresholds to minimize false positives
3. **A/B Testing**: Implement a system to A/B test primary vs fallback models
4. **Performance Attribution**: Build a system that attributes P&L to fair value signals
5. **Decay Forecasting**: Can you predict future model decay based on current trends?